DELTACACHE: LAYER-AWARE KV CACHE MANAGEMENT FOR SCALING POST-TRAINING LLM INFERENCE

Anonymous authors
Paper under double-blind review

ABSTRACT

Post-training LLM workflows—RLHF rollouts, evaluation suites, RAG-augmented serving—involve repeated inference over shared prefixes, making KV cache management a significant performance bottleneck. We present DELTACACHE, a layer-aware KV cache management system that exploits *reuse heterogeneity* across transformer layers: late layers encode task-specific semantics and are more valuable to retain than early layers that capture generic syntactic patterns. DELTACACHE assigns sigmoid-based importance weights to each layer and refines eviction decisions with attention scores. On Mistral-7B at fp16 precision, DELTACACHE achieves **5.06**× average end-to-end speedup (up to **10.90**× on long-prefix workloads) while preserving numerical equivalence with full recomputation. Under memory pressure, layer-aware eviction achieves **66% higher cache hit rates** than LRU. We characterize the conditions under which layer-aware caching is effective and where it provides limited benefit.

1 INTRODUCTION

Post-training—including RLHF, DPO, and iterative refinement—has become a compute-intensive phase of LLM development (OpenAI, 2023; Touvron et al., 2023). The inference workloads it produces share a distinctive structure: many requests reuse the same prefix. In RLHF, multiple rollouts are conditioned on the same prompt; in evaluation pipelines, every checkpoint is tested against the same benchmark suite with shared few-shot demonstrations; in RAG-augmented serving, the same retrieved documents are queried repeatedly. These shared prefixes make the KV cache—which stores $O(2Lnd_k)$ entries for L layers, n tokens, and key dimension d_k—a natural target for reuse.

Modern serving systems like vLLM (Kwon et al., 2023) and SGLang (Zheng et al., 2023) already support prefix caching, but their eviction policies treat all cached entries uniformly. When memory pressure forces eviction, LRU and LFU discard entries without considering *which transformer layer* produced them. This is a missed opportunity. Transformer layers specialize functionally: early layers capture local syntactic patterns, while late layers encode task-specific semantics that directly influence predictions (Rogers et al., 2020; Elhage et al., 2021; Geva et al., 2023). Evicting a late-layer entry is more costly than evicting an early-layer one, because the semantic representation it encodes is harder to reconstruct and more immediately relevant to output quality.

We present DELTACACHE, a KV cache management system that exploits this *layer-wise reuse heterogeneity*. The core idea is to assign each layer an importance weight via a sigmoid function that reflects the syntactic-to-semantic transition:

$$w_l = \frac{1}{1 + e^{-k(l/L - \tau)}} \tag{1}$$

where l is the layer index, L is the total number of layers, k=5 controls the steepness of the transition, and τ=0.3 sets the midpoint. Under this weighting, late layers receive roughly $4.9\times$ higher retention priority than early layers, so that when eviction is necessary, generic early-layer entries are discarded first.

Built around this eviction policy, DELTACACHE integrates prefix-tree lookup, incremental KV computation, and hierarchical GPU/CPU memory management into a complete caching system. On

Algorithm 1 Layer-Aware Cache Eviction

Require: Cache entries E, capacity C, params (k, τ)
while $|E| > C$ **do**
  for each $e \in E$ **do**
    $w_l \leftarrow \sigma\big(k \cdot (l_e/L - \tau)\big)$ ▷ Layer weight
    $I(e) \leftarrow w_l \cdot (w_a \bar{\alpha}_e + w_f \log(1+f_e) + w_r/(1+\log(1+t_e)))$
  end for
  Remove $\arg\min_e I(e)$ from E
end while

Mistral-7B at fp16 precision, it achieves $5.06\times$ average speedup with numerical equivalence to full recomputation. Under memory pressure, layer-aware eviction achieves 66% higher hit rates than LRU. We also characterize the deployment boundary: the system is most effective with long shared prefixes (>500 tokens) and high query volume, and provides limited benefit for short or rapidly-growing contexts.

2 METHOD

2.1 LAYER-AWARE CACHE PRIORITIZATION

The sigmoid weight (Eq. 1) produces a smooth transition: early layers ($l/L < 0.3$) receive weight $\approx$0.18 and late layers ($l/L > 0.7$) receive $\approx$0.88. Within the same layer, we further differentiate entries using attention scores. The composite importance of a cache entry e at layer l is:

$$I(e) = w_l \cdot \big(w_a \cdot \bar{\alpha}_e + w_f \cdot \log(1+f_e) + w_r/(1+\log(1+t_e))\big) \tag{2}$$

where $\bar{\alpha}_e$ is the exponential moving average of attention scores received by entry e, f_e is its access frequency, t_e is the time since last access, and $(w_a, w_f, w_r) = (0.5, 0.3, 0.2)$. Because w_l enters multiplicatively, the layer-level signal dominates: a high-attention entry in an early layer will still be evicted before a moderate-attention entry in a late layer. This enforces a clear hierarchy where layer position determines the primary eviction order, and within-layer attention provides a secondary tiebreaker.

The parameters k and τ do not require careful tuning. Over a grid of $k \in \{1, 3, 5, 7, 10\}$ and $\tau \in \{0.1, \ldots, 0.5\}$ (25 configurations), we observe <1% variation in hit rate, suggesting that performance depends on the general principle of layer-aware prioritization rather than on the specific shape of the weighting function.

2.2 SYSTEM ARCHITECTURE

The eviction policy sits within a broader caching system. Cached KV states are organized in a **prefix tree** (trie), where each root-to-node path corresponds to a token sequence and common prefixes share tree nodes. Lookup takes $O(n)$ time for a sequence of length n.

When a query matches a cached prefix of length m, DELTACACHE performs **incremental computation**: it reuses the cached KV states for positions 1 through m and computes only the remaining tokens $m{+}1$ through n. This is exact for causal language models, since KV states at position i depend only on tokens $[1, \ldots, i]$. To ensure correctness with Rotary Position Embeddings (Su et al., 2024), we explicitly pass absolute position IDs when computing the delta.

A two-tier **GPU/CPU memory hierarchy** manages capacity. Entries are proactively offloaded to CPU memory when GPU utilization exceeds 80%, and predictively prefetched back based on access patterns.

3 EXPERIMENTS

We evaluate on TinyLlama-1.1B (22 layers) and Mistral-7B (32 layers, fp16) using $2\times$ NVIDIA RTX 3090 GPUs. Baselines include full recomputation, LRU, LFU, and attention-only eviction (without layer weighting).

Table 1: Speedup on Mistral-7B (fp16). *Cached* excludes the first (cold) query. Results averaged over 2 runs × 15 queries.

Scenario	Prefix	Speedup	Cached	Reuse
Code Completion	1448	**6.62**×	**10.90**×	92.3%
RAG: API Docs	646	5.90×	6.39×	92.4%
RAG: ML Tutorial	647	5.56×	6.46×	92.3%
Long System Prompt	484	3.83×	4.74×	92.2%
Few-shot Sentiment	282	3.39×	3.93×	89.4%
Average	–	**5.06**×	**6.48**×	91.7%

Table 2: Cross-model validation on TinyLlama-1.1B (2 runs × 20 queries). The lower speedup reflects the smaller per-token computation cost of a 1.1B-parameter model.

Scenario	Prefix	Speedup	Reuse
Code Completion	1464	**3.64**×	93.8%
RAG: API Docs	660	3.94×	94.4%
RAG: ML Tutorial	656	2.96×	94.2%
Long System Prompt	497	1.80×	94.0%
Few-shot Sentiment	298	1.22×	90.8%
Average	–	**2.60**×	93.4%

3.1 Correctness Validation

Because cached KV states are reused verbatim (not approximated), the outputs should be numerically equivalent to full recomputation up to floating-point precision. We verify this by computing $\|\text{kv}_{\text{cached}} - \text{kv}_{\text{fresh}}\|_\infty$ over 100 test samples per model. At fp16 precision, all samples for both TinyLlama and Mistral-7B fall within 0.02 (max observed: 0.0156), consistent with the expected rounding behavior of half-precision arithmetic. The 8-bit quantized variant of Mistral-7B shows larger deviations (max 0.60), but these arise from quantization noise rather than from the caching mechanism itself.

3.2 End-to-End Speedup

We measure end-to-end latency on five workloads that exhibit the repeated-prefix structure common in post-training: code completion with a large fixed context (∼1450 tokens), RAG over fixed knowledge bases (∼650 tokens), few-shot evaluation (∼280 tokens), and system-prompt-guided generation (∼480 tokens). Each scenario is run with 15 queries (Mistral-7B) or 20 queries (TinyLlama) averaged over 2 runs.

Tables 1 and 2 summarize the results. Mistral-7B averages $5.06\times$ speedup across the five scenarios, with code completion reaching $10.90\times$ on cached queries. TinyLlama shows a consistent but smaller gain ($2.60\times$ average), which is expected: the per-token computation cost of a 1.1B model is lower, so the relative savings from cache reuse are smaller. Across both models, speedup correlates strongly with prefix length—the avoided $O(m^2)$ attention computation over m cached prefix tokens dominates the benefit—and token reuse rates exceed 90% in all scenarios.

3.3 Eviction Policy Under Memory Pressure

The speedup results above assume sufficient memory to hold the working set. When cache capacity is limited, the eviction policy becomes the determining factor. To isolate this effect, we run 320 queries across 40 unique prefixes on TinyLlama-1.1B and restrict cache capacity to 10–50% of the working set, forcing frequent eviction.

Table 3 shows that layer-aware eviction outperforms both baselines at every capacity level. The gap is largest at 10% capacity, where layer-aware eviction achieves a 66% higher hit rate than LRU (25.9% vs. 15.6%). This is the regime most relevant to post-training: during RLHF, GPU memory is

Table 3: Cache hit rates under memory pressure (TinyLlama-1.1B, 40 prefixes, 320 queries).

Policy	10% cap.	20% cap.	30% cap.	50% cap.
LRU	15.6%	31.6%	42.8%	62.5%
LFU	20.9%	42.2%	51.6%	65.6%
Layer-Aware	**25.9%**	**43.1%**	**55.3%**	**69.7%**
Δ vs. LRU	**+66%**	+36%	+29%	+12%

Table 4: Context length scaling (TinyLlama-1.1B). Cached latency stays nearly constant while baseline cost grows with context length.

Context	Baseline (ms)	Cached (ms)	Speedup
501	35.2	17.6	2.00×
1001	54.3	16.3	3.32×
1501	69.8	16.7	4.18×
1801	90.0	17.4	**5.17**×

shared between model weights, optimizer states, gradients, and KV cache, leaving limited headroom for caching. As capacity increases and eviction becomes less frequent, the policies converge—at 50%, even LRU reaches 62.5%. The pattern is intuitive: when the system can only retain a small fraction of the working set, choosing *which* entries to keep matters far more than when most entries fit anyway.

3.4 Context Length Scaling and Memory Overhead

Table 4 illustrates a characteristic property of incremental computation: the cached query latency stays nearly constant at $\sim$17 ms regardless of context length, because only the short suffix is recomputed. Baseline latency, by contrast, grows linearly with context. The practical implication is that speedup improves with longer contexts—roughly $2.4\times$ per additional 1K tokens—making DeltaCache particularly attractive for the long-context settings increasingly common in post-training.

In terms of memory cost, DeltaCache adds 17–22% GPU overhead (505 MB for TinyLlama, 2.4 GB for Mistral-7B), covering KV tensor storage, prefix tree structure, and importance metadata. For Mistral-7B, each additional GB of cache memory translates to approximately $2.2\times$ speedup.

4 Discussion and Limitations

The results above paint a consistent picture: DeltaCache is effective when the workload involves a stable, long prefix shared across many queries, and the available cache memory is insufficient to hold the entire working set. These conditions arise naturally in post-training. During RLHF or DPO, the policy model generates many completions for the same prompt, each sharing an identical prefix; during evaluation, every model checkpoint is tested on the same benchmark suite with shared few-shot demonstrations; in RAG serving, the same retrieved passages are queried by multiple users. In each case, the prefix is long enough and reused often enough for caching to pay off.

The system does not help in all settings. When prefixes are short ($<$200 tokens), cache management overhead exceeds the saved computation, resulting in a net slowdown (we measure $0.68\times$ at 100 tokens). Multi-turn conversations with growing context also see little benefit ($0.90\times$), because each turn appends substantial new content and the prefix reuse fraction drops. Our layer weighting function is grounded in the layer specialization pattern documented for decoder-only transformers (Rogers et al., 2020); architectures with different depth profiles (e.g., encoder-decoder models) may require a different parameterization of τ. Finally, the current implementation targets single-request latency and does not address batched inference; integrating layer-aware eviction into production serving systems like vLLM (Kwon et al., 2023) or SGLang (Zheng et al., 2023) would require coordinating eviction decisions across concurrent requests.

We note that layer-aware eviction is orthogonal to both the prefix matching mechanism and the token-level eviction dimension. It could be combined with token-level strategies such as H2O (Zhang et al., 2024) or with KV cache quantization methods like KIVI (Liu et al., 2024), jointly optimizing along the layer, token, and precision axes.

5 CONCLUSION

We have presented DELTACACHE, a KV cache management system that uses layer-aware importance weighting to decide which cache entries to retain under memory pressure. The approach is grounded in a well-established property of transformers—that late layers encode task-specific semantics while early layers capture more generic patterns—and translates it into a simple, tuning-free eviction policy. On Mistral-7B, DELTACACHE achieves $5.06\times$ average speedup with numerical equivalence to full recomputation, and under tight memory constraints it retains 66% more useful cache entries than LRU. The system is most effective for the fixed-prefix, high-reuse workloads that characterize post-training—RLHF rollouts, evaluation pipelines, and RAG serving—and we have identified the conditions under which simpler alternatives suffice.

REFERENCES

Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021.

Mor Geva, Jasmijn Bastings, Katja Filippova, and Amir Globerson. Dissecting recall of factual associations in auto-regressive language models. *arXiv preprint arXiv:2304.14767*, 2023.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, pp. 611–626, 2023.

Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024.

OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

Anna Rogers, Olga Kovaleva, and Anna Rumshisky. A primer in bertology: What we know about how bert works. *Transactions of the Association for Computational Linguistics*, 8:842–866, 2020.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhiyuan Wang, and Beidi Chen. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36, 2024.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kober, Ying Sheng, Joseph E Gonzalez, Ion Stoica, and Hao Zhang. Efficiently programming large language models using sglang. *arXiv preprint arXiv:2312.07104*, 2023.